ADVANCED PYTHON PROGRAMMING

PROJECT WRITE-UP – ENCHANTED PAGES: ENCRYPTED DIARY

NAME: G K MONIKA

REG.NO: 23MID0002

1. Title

Enchanted Pages – A Secure Digital Diary with Encryption and Telegram Integration

2. Motivation

People often use notes applications or social media drafts to save their personal thoughts, memories, and daily experiences. However, these platforms may not provide the personal diary experience or the level of privacy that users expect.

I wanted to build something that feels like an actual diary but works digitally. The idea was to allow users to maintain multiple diaries, protect them with individual passwords, and securely store their personal entries.

I also wanted to make writing more convenient, which led to the integration of a Telegram bot that allows users to save their thoughts directly through Telegram.

3. Existing Gap and Project USP

Most basic diary and note-taking applications focus on writing and organising content. Some provide password protection, while others offer convenient ways to capture notes.

The idea behind Enchanted Pages is to bring these features together.

Key USPs:

- Multiple diaries can be maintained under one user account.
- Individual diaries are password-protected.
- Entry titles and content are encrypted before database storage.
- Users can save notes through a Telegram bot.
- Telegram notes can be selectively imported into a chosen diary.
- A fantasy-inspired interface provides a more personal journaling experience.

The main difference is the combination of privacy, organisation, and convenient note capture in one application.

4. Architecture

The application follows a modular Flask-based architecture.

Architecture components:

1. Frontend: Users interact with the application through a web browser. The interface handles registration, login, diary management, and entry operations.
2. Flask Application (app.py): Handles HTTP requests, authentication, form validation, diary operations, and communication between modules.
3. Encryption Module (crypto.py): Uses PBKDF2-HMAC-SHA256 for key derivation and Fernet for encryption and decryption.
4. Database Layer (database.py): Handles PostgreSQL connections and database operations.
5. PostgreSQL Database: Stores user accounts, diaries, encrypted entries, unlock sessions, and Telegram-related information.
6. Telegram Integration (telegram_integration.py): Processes bot commands, manages linked accounts, and handles saved messages.
7. Telegram Bot API: Connects the Telegram application to the Flask backend through webhooks.

Application workflow:

User → Web Application → Flask Backend → Authentication and Encryption → PostgreSQL Database → Response to User.

For Telegram:

Telegram User → Telegram Bot API → Flask Webhook → Telegram Integration Module → Database → Import into Selected Diary.

5. Complete Tech Stack Information

Component	**Technology**	**Purpose**
Programming language	Python	Main application development
Backend	Flask	Routing and server-side logic
Frontend	HTML, CSS, JavaScript	User interface
Template engine	Jinja2	Dynamic HTML rendering
Database	PostgreSQL	Persistent data storage
Database hosting	Neon	Hosted PostgreSQL

Database connector	psycopg2	Python-PostgreSQL communication
Encryption	Fernet	Entry encryption and decryption
Key derivation	PBKDF2-HMAC-SHA256	Deriving encryption keys
Password security	Werkzeug	Password hashing and verification
API integration	Telegram Bot API	Telegram communication
HTTP requests	Requests	Telegram API communication
Deployment	Vercel	Application hosting
Configuration	Environment variables	Managing secrets and deployment settings

6. Datasets Used

No external datasets were used in this project.

The application works with user-generated data, including:

- User registration information.
- Diary names and passwords.
- Diary entry titles and content.
- Telegram account linking information.
- Notes saved through the Telegram bot.

The application stores and manages this information through its PostgreSQL database.

7. Prompts Used for Website Construction

ChatGPT was used as a development assistant during different stages of the project. The following are the main prompts used, reconstructed to represent the requirements given during development.

Prompt 1 – Initial Application Development

"Build a Python Flask diary application where users can register, log in, create multiple diaries, and protect each diary with an individual password. Users should be able to create, edit, and view entries. Encrypt diary content before storing it in the database."

Prompt 2 – Diary UI Redesign

"Redesign the existing application interface to look and feel like an actual diary or journal instead of a regular web application. Keep the backend, database, and existing functionality unchanged."

Prompt 3 – Fantasy Theme and Visual Improvements

"Give the application a richer fantasy-inspired appearance. Improve the colours, typography, icons, and diary wallpapers. Make the pages visually consistent while retaining all existing features."

Prompt 4 – Telegram Bot Integration

"Integrate a Telegram bot into the existing diary application so that every registered user can connect their own Telegram account. Allow users to save notes, view pending saved messages, unlink their accounts, and import selected notes into their diaries."

Prompt 5 – Telegram API and Deployment

"Help configure the Telegram Bot API integration, webhook, environment variables, and deployment. Ensure that Telegram commands are processed by the Flask backend and that saved messages can be securely imported into the application."

Prompt 6 – Application Improvements

"Review the application's current limits on diaries, entries, and content length. Identify missing input validation and suggest improvements such as pagination and performance optimisation."

These prompts were used iteratively, with changes and corrections made based on the application's behaviour during development.

8. API Integration

The project integrates the Telegram Bot API to allow users to save diary notes without directly opening the website.

The integration works through a webhook-based communication system.

Integration workflow:

1. The user opens the Telegram integration page on the website.
2. The application generates a temporary linking code.
3. The user sends /link CODE to the Enchanted Pages Telegram bot.
4. The backend verifies the code and links the Telegram account.
5. The user can now send notes using the /save command.
6. The bot stores the note as an encrypted pending item.
7. The user opens the website, unlocks a diary, and imports the selected note.

Telegram commands:

Command	Function
/start	Starts interaction with the bot
/help	Displays available commands
/link CODE	Connects a Telegram account
/save	Saves a note
/saved	Checks pending saved notes

/unlink Disconnects the Telegram account

The Telegram Bot API communicates with the Flask application through HTTPS webhooks. A webhook secret is used to validate incoming requests.

9. Users, Inputs and Outputs

Users

The application is designed primarily for individual users who want to maintain private digital diaries.

Each registered user has access to their own diaries and entries. Users can also connect their Telegram accounts to save notes.

Inputs

Input	Type / Format	Purpose
Username	Text	User identification
Email	Email address	Account registration
Account password	Password	User authentication
Diary name	Text	Creating a diary
Diary password	Password	Protecting and unlocking a diary
Entry title	Text	Identifying an entry
Entry content	Text	Storing personal thoughts and memories
Telegram linking code	Alphanumeric code	Connecting Telegram accounts
Telegram message	Text	Saving notes through the bot

Processing

The application processes inputs through Flask routes and the relevant modules.

- User credentials are validated during registration and login.
- Diary passwords are verified before protected content is accessed.
- Entry titles and content are encrypted before database storage.
- Telegram commands are received through webhooks and processed by the Telegram integration module.
- Selected Telegram notes are imported into the user's chosen diary.

Outputs

- User dashboard containing the user's diaries.
- Password-protected diary pages.
- Decrypted diary entries after successful unlocking.
- Confirmation messages for diary and entry operations.
- Telegram bot responses for supported commands.
- Pending Telegram notes available for import.
- Successfully imported notes appearing as diary entries.

10. References

1. Flask Documentation – https://flask.palletsprojects.com/
2. Python Documentation – https://docs.python.org/3/
3. PostgreSQL Documentation – https://www.postgresql.org/docs/
4. Neon Documentation – https://neon.com/docs
5. Cryptography Documentation (Fernet) – https://cryptography.io/en/latest/fernet/
6. PBKDF2 Documentation – https://cryptography.io/en/latest/hazmat/primitives/key-derivation-functions/
7. Telegram Bot API Documentation – https://core.telegram.org/bots/api
8. Vercel Documentation – https://vercel.com/docs
9. Werkzeug Documentation – https://werkzeug.palletsprojects.com/

Project Summary: Enchanted Pages combines a personal journaling experience with password-based protection, encrypted storage, and Telegram integration. The objective was not just to create another notes application, but to build a space where users can comfortably organise and preserve their thoughts.